VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



PARALLEL COMPUTING

Assignment 1

Mutex Lock, Sense-Reversing Barrier và Concurrent Counter

Cài đặt, kiểm tra tính đúng đắn và nghiên cứu hiệu năng

Advisor: Assoc. Prof. Trần Văn Hoài
PhD. Đặng Thanh Điệp
Students: Họ và tên thành viên 1 - MSHV.
Họ và tên thành viên 2 - MSHV.
Họ và tên thành viên 3 - MSHV.

HO CHI MINH CITY, OCTOBER 2026

Contents

1 Member list & Workload **2**

2 Giới thiệu và câu hỏi nghiên cứu **2**

3 Nền tảng thí nghiệm **3**

4 Cài đặt **4**

4.1 Mutex lock 4

4.2 Sense-reversing barrier 5

4.3 Concurrent counter 6

5 Kiểm tra tính đúng đắn **6**

5.1 Chiến lược 6

5.2 Mutex lock 7

5.3 Sense-reversing barrier 7

5.4 Concurrent counter 7

5.5 Mutation testing 8

5.6 Giới hạn của phương pháp 8

6 Phương pháp đo **9**

7 Kết quả và phân tích **10**

7.1 Mutex lock 10

7.2 Barrier 15

7.3 Concurrent counter 16

7.4 So sánh giữa các máy 18

8 Các yếu tố đe dọa độ tin cậy **18**

9 Kết luận **19**

References **21**

A Cách build và chạy lại **21**

B Cấu trúc mã nguồn **22**

C Khai báo sử dụng công cụ AI **22**

1 Member list & Workload

No.	Fullname	Student ID	Problems	Percentage of work
1	...	...	- ...	...%
2	...	...	- ...	...%
3	...	...	- ...	...%

[Cần viết: nhóm điền bảng phân công: họ tên, MSHV, phần việc (cài đặt, kiểm tra, đo trên máy nào, phân tích, viết báo cáo) và tỉ lệ đóng góp.]

2 Giới thiệu và câu hỏi nghiên cứu

Nhóm cài đặt bằng C++ mutex lock (TAS, TTAS, MCS), sense-reversing barrier và concurrent counter (lock, CAS, fetch-and-add), kiểm tra tính đúng đắn, và đo khả năng mở rộng trên máy shared-memory, so với cấu trúc của thư viện. Phần 4 (MPI-3 RMA [6]) không bắt buộc và nhóm không làm.

- **Cache line.** Đơn vị 128 byte mà phần cứng giữ nhất quán giữa các core. Nhiều core cùng đọc được, nhưng ghi thì một core phải giữ line độc quyền và làm mất hiệu lực bản sao ở core khác.
- **Tranh chấp.** Nhiều thread cùng muốn vào critical section. `OUTSIDE_WORK` $= 0$ là tranh chấp tối đa.
- **TAS, TTAS, MCS.** TAS lặp `exchange` lên cờ lock. TTAS đọc cờ tới khi rảnh rồi mới `exchange` [1]. MCS xếp hàng, mỗi thread chờ trên cờ riêng [5].
- **Episode.** Một lần cả n thread cùng vượt barrier [2].
- **FAA, CAS.** Fetch-and-add cộng và trả giá trị cũ trong một bước. Compare-and-swap chỉ ghi nếu giá trị chưa đổi, nếu không thì thất bại và phải thử lại.

Dự đoán dưới đây dựa trên một mô hình. Dưới tranh chấp, chi phí chủ yếu là số lần một cache line đi giữa các core trên đường tới hạn [3]. Trên `apple-m5`, điểm gãy dự kiến ở 4 thread (hết P-core) và 10 thread (hết core).

RQ1 (đúng đắn). Các cài đặt có đúng không, và bộ kiểm tra có đủ nhạy để bắt lỗi cài đặt? D1.1: stress test bắt lỗi thuật toán; lỗi memory order quá yếu chỉ lộ thỉnh thoảng trên ARM, cần ThreadSanitizer; lỗi treo cần watchdog.

RQ2 (lock: mở rộng). TAS, TTAS, MCS mở rộng thế nào theo số thread, `OUTSIDE_WORK` và N, so với lock thư viện? D2.1: TAS chậm nhất (mỗi lần thử là một phép ghi), TTAS ở giữa ("bầy đàn" mỗi lần nhả), MCS nhanh nhất khi chưa vượt số core (chuyển giao bằng một phép ghi). D2.2: khác biệt lớn nhất khi critical section ngắn. D2.3: lock bão hoà khi $p > 1 + \mathrm{OW}$. D2.4: vượt số core thì MCS sụp nặng nhất (lock giao cho thread đang không chạy), lock thư viện không sụp.

RQ3 (lock: backoff và công bằng). Backoff giúp hay hại, ở chế độ nào; lock nhanh nhất có công bằng không? D3.1: backoff giúp TAS hơn TTAS. D3.2: `sleep_for` là system call nên ngủ lâu hơn yêu cầu; khi $\mathrm{OW} = 0$ tổng thời gian không xấu đi nhưng mất công bằng, khi $\mathrm{OW} > 0$ thì chậm hơn. D3.3: MCS công bằng (FIFO); TAS, TTAS thiên vị thread vừa nhả lock.

RQ4 (barrier). Chi phí một episode tăng thế nào theo số thread, theo cách chờ, so với barrier thư viện? D4.1: p phép `fetch_add` nối tiếp trên một line, nên số episode mỗi giây $\propto 1/p$. D4.2: barrier

spin sụp khi vượt số core; barrier chờ chặn chỉ thắng ở đó. D4.3: backoff làm barrier chậm khi mỗi thread có core riêng.

RQ5 (counter). Thứ tự hiệu năng giữa FAA, CAS (có/không backoff) và counter dùng lock, và vì sao? D5.1: FAA $<$ CAS $<$ lock về thời gian, không cách nào mở rộng. D5.2: số lần CAS hỏng mỗi lần tăng tăng theo p. D5.3: backoff giảm CAS hỏng khi không có việc giữa hai lần tăng. D5.4 (giả thuyết): `seq_cst` gần như không đắt hơn `relaxed`.

RQ6 (giữa các máy). Các kết luận trên có giữ nguyên trên máy khác không? D6.1: thứ tự giữ nguyên, điểm gãy dời theo số core.

RQ	Dự đoán	Thí nghiệm	Hình, bảng
RQ1	D1.1	stress, TSan, mutation	Bảng 3
RQ2	D2.1–D2.4	lock × thread × OW × N	Hình 1, 2; Bảng 6
RQ3	D3.1–D3.3	sáu biến thể backoff, thời gian ngủ thật	Hình 3, 4
RQ4	D4.1–D4.3	episode/giây, bảy barrier	Hình 5; Bảng 7
RQ5	D5.1–D5.4	thời gian, số CAS hỏng	Hình 6, 7; Bảng 8
RQ6	D6.1	cùng quy trình trên mỗi máy	Bảng 2

Bảng 1: Câu hỏi nghiên cứu, dự đoán, thí nghiệm và hình, bảng trả lời.

Đóng góp của báo cáo:

- cài đặt ba nhóm cấu trúc kèm lập luận memory order cho từng thao tác atomic;
- bộ kiểm tra (stress, watchdog, ThreadSanitizer, schedule fuzzing) được đánh giá bằng mutation testing;
- quy trình đo một lệnh, sinh lại mọi hình và bảng, thêm máy không cần sửa mã;
- so sánh số đo với cận dưới lý thuyết và với dự đoán viết trước khi đo.

Mục Nền tảng thí nghiệm mô tả máy đo, và mục Cài đặt trình bày thuật toán. Mục Kiểm tra tính đúng đắn trả lời RQ1. Mục Phương pháp đo mô tả cách đo, và mục Kết quả và phân tích trả lời RQ2–RQ6. Báo cáo kết thúc bằng mục Các yếu tố đe dọa độ tin cậy, mục Kết luận và phụ lục.

3 Nền tảng thí nghiệm

Đề yêu cầu số nền tảng shared-memory không ít hơn số thành viên. Báo cáo này có số đo của 1 máy, `apple-m5`. Mỗi thành viên thêm máy của mình bằng cách chạy `make bench` trên máy rảnh, gửi lại thư mục `results/<tên máy>/` và thêm tên máy vào danh sách máy của báo cáo. Bảng cấu hình (Bảng 2) và các hình của máy mới được sinh lại tự động. Mọi nhận định về hiệu năng dưới đây chỉ nói về `apple-m5`.

[Cần viết: nhóm có mấy thành viên, và máy của các thành viên còn lại (sau khi họ chạy make bench)]

Máy `apple-m5` là một Apple M5 với 10 core không đồng nhất, gồm 4 P-core (performance) và 6 E-core (efficiency). Mỗi cụm có cache L2 riêng. Máy không có SMT. Ba đặc điểm của máy ảnh hưởng tới kết quả. Thứ nhất, khi số thread vượt 4, một phần thread chạy trên E-core chậm hơn, và cache line dùng chung phải đi qua lại giữa hai cụm. Thứ hai, macOS không cho gắn thread vào một core cố định, nên loại core của mỗi thread do scheduler quyết định và có thể đổi giữa các lần chạy. Nhóm giảm biến thiên này bằng cách lặp lại và lấy trung vị. Thứ ba, AArch64 có memory model yếu hơn x86-TSO. Các memory order khác nhau sinh ra các lệnh máy khác nhau (store `relaxed` là `str`, store `release` là `stlr`). Vì vậy lỗi chọn sai memory order có thể quan sát được trên máy này, trong khi trên x86 nó thường không bao giờ lộ ra. CPU có các lệnh atomic LSE của ARMv8.1, nên `fetch_add` và `compare_exchange` là một lệnh máy (`ldadd`, `cas`) chứ không phải vòng LL/SC.

Hệ điều hành báo cache line, đơn vị dữ liệu mà phần cứng giữ nhất quán giữa các core, dài 128 byte. Mã nguồn đệm mọi biến dùng chung theo hằng `pc::kCacheLine`, bằng 128 trên Apple arm64 và 64 trên nền tảng khác. Nhóm kiểm tra rằng kích thước và căn chỉnh của mọi kiểu lock và của nút hàng đợi MCS đều bằng 128 byte trên máy này.

Phần mềm gồm macOS 27.0.1 và Apple clang 21.0.0 với cờ `-std=c++20 -O2`. Apple clang không nhận `-fopenmp`, và macOS không có `pthread_spinlock_t` hay `pthread_barrier_t`. Vì vậy các baseline trên máy này là `std::mutex`, `pthread_mutex_t`, `os_unfair_lock`, `std::barrier` và barrier dựng từ mutex và condition variable.

Toàn bộ số đo đến từ một đợt đo duy nhất dài 1 giờ 26 phút, gồm 12 152 lần chạy. Mỗi cấu hình có 7 lần lặp, và thứ tự các cấu hình xáo trộn trong từng lượt. Máy cắm sạc, chế độ tiết kiệm điện tắt, và không có tiến trình đo hay build nào khác chạy cùng lúc. Load average một phút lúc bắt đầu là 6,73, trong khoảng thường thấy của máy khi chỉ có ứng dụng nền. Không lần chạy nào sai kết quả kiểm tra. Có 9 lần vượt thời hạn, đều là cấu hình có MCS khi số thread vượt số core, và chúng được đo lại với số vòng lặp nhỏ hơn 100 lần (mục Phương pháp đo).

Bảng 2: Cấu hình các máy dùng để đo. Thông tin do `scripts/machine_info.sh` ghi lại lúc chạy đợt đo; kích thước cache line do hệ điều hành báo.

	`apple-m5`
CPU	Apple M5
Kiến trúc	arm64
Số core	10 (4 P-core + 6 E-core)
Cache line (hệ thống báo)	128 B
Cache line dùng để đệm trong mã	128 B
Cache L1d	128 KiB (P) / 64 KiB (E)
Cache L2	16 MiB per 4 P cores / 6 MiB per 6 E cores
Cache L3	–
RAM	16 GiB
Hệ điều hành	macOS 27.0.1 (26A434)
Nhân	Darwin 27.0.0
Compiler	Apple clang version 21.0.0 (clang-2100.1.1.101)
Cờ biên dịch	`-std=c++20 -O2 -Wall -Wextra -pthread`
OpenMP	không (không có baseline OpenMP)
Lệnh chờ trong vòng spin	`isb`
Ghim thread vào core	không có (macOS trên Apple Silicon không có API ghim thread)
Nguồn điện lúc đo	cắm sạc
Thành viên chạy đo	*(điền tên)*

4 Cài đặt

4.1 Mutex lock

Mọi lock trong `include/lock.h` có đúng `acquire()` và `release()` không tham số như khung của đề. `using pc::lock_mcs;` cho dòng `lock_mcs lock;` biên dịch nguyên văn. Lấy lock dùng acquire, nhả lock dùng release. Cặp này làm mọi phép ghi trong critical section trước nhìn thấy được ở chủ lock sau. Trạng thái lock được khai báo `alignas(kCacheLine)`, nên mỗi lock chiếm riêng một line 128 byte, không false sharing.

TAS lặp `exchange(true, acquire)` (lệnh `swpab`) tới khi đọc được `false`. TAS nhả lock bằng `store(false, release)` (`stlrb`). TTAS chờ bằng `load(relaxed)` (`ldrb`) rồi mới `exchange`. Phép đọc chỉ là gợi ý, `exchange` thắng mới là phép acquire.

MCS [5] giữ `tail_` (`nullptr` là rảnh). Mỗi thread chờ trên cờ `locked` trong nút của mình, nên mỗi lần chuyển giao chỉ ghi một line của một thread, theo thứ tự FIFO. Trong các đoạn mã dưới đây, `relaxed`, `acquire`, `release`, `acq_rel` viết tắt cho `std::memory_order_*`.

```
1 me->next.store(nullptr, relaxed); me->locked.store(true, relaxed);  // not yet reachable
```

```
pred = tail_.exchange(me, acq_rel);                 // release: publish node; acquire: take lock
if (pred == nullptr) return;
pred->next.store(me, release);
while (me->locked.load(acquire)) cpu_relax();       // spin on own cache line
// release(): if next == null, CAS tail_ me -> null (release); on failure wait for next
succ->locked.store(false, release);                 // hand over
```

Hai phép ghi relaxed an toàn vì nút chỉ trở nên với tới được qua hai phép release theo sau. Nếu CAS `tail_` thất bại, một thread đã đổi `tail_` nhưng chưa nối vào `next`. Chủ lock phải chờ nó, vì nếu bỏ đi thì thread đó chờ mãi. Trên AArch64 các thao tác thành `swpal`, `stlr`, `ldaprb`, `casl`, `stlrb`. Nút `mcs_node` cũng chiếm riêng một line.

MCS có hai giới hạn. Nút là biến `thread_local` dùng chung cho mọi `lock_mcs`, vì giao diện không có tham số, nên một thread không giữ được hai lock MCS cùng lúc. Lock được giao cho đúng một thread, nên nếu thread đó bị preempt thì mọi thread phía sau phải chờ scheduler.

`lock_tas_bo<B>` và `lock_ttas_bo<B>` gọi chính sách `B` sau mỗi lần thử thất bại. TTAS chỉ lùi lại khi thua `exchange` (BackoffLock [3]). Ba chính sách là `sleep` theo mẫu của đề (`sleep_for`, `delay` nhân đôi từ 100 ns tới `MAX_DELAY` = 100 000 ns), `spin` (quay ngẫu nhiên, cửa sổ nhân đôi từ 4 tới 1024 lần `cpu_relax`) và `yield` (quay 64 lần rồi `yield`). `sleep_for` là system call. Trên `apple-m5`, yêu cầu 100 ns ngủ trung vị 2 670 ns (nhóm đo bằng `build/sleep_latency`).

Lock thư viện được bọc inline sau cùng giao diện. Đó là `std::mutex`, `pthread_mutex_t`, `os_unfair_lock` (macOS), `pthread_spinlock_t` (Linux), `omp_lock_t` (khi có OpenMP). Vòng spin gọi `cpu_relax()`, tức là `pause` trên x86, `isb` trên AArch64 vì `yield` của Apple gần như không làm gì (0,3 ns so với 12,1 ns của `isb`, cùng phép đo).

4.2 Sense-reversing barrier

Barrier cho n thread bảo đảm không thread nào ra khỏi lần gọi `wait()` thứ k trước khi cả n thread đã gọi lần thứ k, và dùng lại được qua mọi episode. Barrier một cờ "mở/đóng" không dùng lại được. Đóng cờ sớm thì thread chậm chờ mãi, đóng muộn thì thread nhanh qua luôn episode sau. Sense-reversing barrier [2] không đặt lại cờ mà đảo nó mỗi episode. Mỗi thread chờ cờ bằng *local sense* của mình, nên hai episode kề nhau chờ hai giá trị khác nhau.

```
const bool my_sense = local_[tid].sense;
if (count_.fetch_add(1, acq_rel) == n_ - 1) {       // last arriver
    count_.store(0, relaxed);                       // 1. reset for the next episode
    sense_.store(my_sense, release);                // 2. THEN release the others
} else {
    while (sense_.load(acquire) != my_sense) backoff.pause();
}
local_[tid].sense = !my_sense;
```

Thread tới cuối phải đặt lại bộ đếm trước khi đảo cờ. Nếu đảo trước, thread vừa được thả có thể tới episode sau, rồi bị phép đặt lại muộn xoá mất lần tới, và mọi thread chờ mãi. Bài stress thường bắt biến thể đảo thứ tự này trong 2 trên 5 lần chạy, còn bài có schedule fuzzing bắt cả 5 lần.

`fetch_add` cần acq_rel (`ldaddal`). Nửa release công bố phép ghi trước barrier, nửa acquire cho thread tới cuối thấy phép ghi của mọi thread khác. Với chỉ release (`ldaddl`), bài stress trên `apple-m5` bắt được thread tới cuối đọc dữ liệu cũ (1 trong 5 lần chạy). Phép đặt lại relaxed đủ vì nó đứng trước phép ghi release lên `sense_` (`stlrb`), mà thread chờ đọc bằng acquire (`ldaprb`).

`count_`, `sense_` và mỗi local sense nằm trên các cache line riêng, để lần tới không làm mất hiệu lực line đang được chờ. Local sense nằm trong barrier, đánh chỉ số bằng `tid`, vì một biến `thread_local` sẽ bị hai barrier dùng xen kẽ làm hỏng.

Các cách chờ gồm quay (`sr`), ba chính sách backoff như lock, và chờ chặn (`sr_block`) bằng `std::atomic::wait` và `notify_all`. Các barrier thư viện gồm `std::barrier`, barrier dựng từ mutex và condition variable (`condvar`), `pthread_barrier_t` (Linux) và `omp barrier`. Các giới hạn là số thread cố định, barrier tập trung tốn $O(n)$ trên đường tới hạn, và biến thể quay cần mỗi

thread một core.

4.3 Concurrent counter

Counter bắt đầu từ 0. `increment()` cộng 1 và trả về giá trị *trước* khi tăng. M lần tăng phải trả về đúng $0, \ldots, M-1$, mỗi giá trị một lần, và theo thứ tự thời gian thực (linearizable [4]). Ba cài đặt là `counter_lock<L>` (biến `long` trong critical section của mỗi lock ở phần 1), `counter_cas` (lock-free, có biến thể backoff) và `counter_faa` (wait-free). Trên `apple-m5` có 19 counter.

```
long old = value_.load(relaxed);                    // a guess; the CAS checks it
while (!value_.compare_exchange_weak(old, old + 1, Order, relaxed)) {
    ++failed_;                                      // thread_local, failure path only
    backoff.pause();
}
return old;
// fetch-and-add: return value_.fetch_add(1, Order);
```

Thứ tự mặc định là `relaxed`, vì tính atomic của phép đọc-sửa-ghi và thứ tự toàn phần của mọi phép ghi lên một biến atomic đúng với mọi memory order. Counter cũng không bảo vệ dữ liệu nào khác. Biến thể `faa_seqcst`, `cas_seqcst` đo cái giá của `seq_cst`. Counter dùng lock không có atomic riêng. Nó đúng nhờ cặp release–acquire của lock.

Apple clang bật LSE (lệnh atomic đơn của ARMv8.1) mặc định. `fetch_add` là một lệnh `ldadd` (`ldaddal` với `seq_cst`), không vòng lặp. CAS là một lệnh `cas` (`casal`), chỉ hỏng khi giá trị thật sự đã đổi, nên mỗi lần `cas_fail` là một xung đột thật. Khi tắt LSE, cả hai thành `ldxr`/`stxr` (LL/SC), và CAS weak có thể hỏng giả. Trên x86-64 hai thứ tự sinh cùng một lệnh (`lock xadd`, `lock cmpxchg`).

Giá trị atomic nằm riêng một cache line. Trong `counter_lock`, lock và giá trị nằm trên hai line để thread chờ không làm mất hiệu lực line của giá trị. Vì vậy mỗi lần tăng ghi lên hai line. Số lần CAS hỏng được đếm trong biến `thread_local` để không thêm một line bị tranh chấp. Về giới hạn, counter CAS chỉ lock-free nên một thread có thể bị đói. `counter_lock` mang theo giới hạn của lock bên trong.

5 Kiểm tra tính đúng đắn

5.1 Chiến lược

Nhóm kiểm hai nhóm tính chất. Tính an toàn (safety) nghĩa là không có hai thread cùng ở trong critical section, không thread nào ra khỏi barrier sớm, không giá trị counter nào bị trả về hai lần. Tính sống còn (liveness) nghĩa là chương trình không treo. Công bằng chỉ được đo.

Cùng một bài kiểm tra dạng template chạy cho mọi cài đặt, kể cả lock và barrier của thư viện. Nhóm dùng bốn kỹ thuật:

- stress test với 1 thread, số luồng phần cứng và gấp đôi số đó (trên `apple-m5`: 1, 10, 20), để thread bị tước core giữa giao thức;
- watchdog, thread giám sát báo `FAIL` khi bài kiểm tra ngừng tiến triển;
- ThreadSanitizer (TSan), công cụ của compiler báo hai truy cập không atomic xung đột mà không có quan hệ happens-before; atomic riêng của bài kiểm tra dùng `relaxed` để không tự tạo thứ tự;
- mutation testing, chạy bộ kiểm tra trên các biến thể cố ý làm hỏng.

Barrier dùng thêm schedule fuzzing. Counter dùng thêm phép kiểm thứ tự thời gian thực.

Trên `apple-m5`, `make test` và `make tsan` đạt cho 12/12 lock, 7/7 barrier và 19/19 counter, và TSan không báo data race nào. `make mutation` bắt cả 14 biến thể hỏng. Đầu ra nằm trong `results/<máy>/`.

[Cần viết: kết quả make test, make tsan, make mutation trên máy của các thành viên còn lại (nhất là một máy x86-64 và một máy Linux).]

5.2 Mutex lock

Trong critical section, bài kiểm tra tăng một biến đếm không atomic (đọc, cộng một, ghi) và đếm số thread đang ở bên trong. Vi phạm hiện ra trực tiếp, hoặc qua một lần cập nhật bị mất. Nhóm chạy phép kiểm với số vòng cố định (1, 2 và 10 thread) và trong 300 ms với 4, 10 và 20 thread. Lượt 20 thread thử lock khi quá tải. Watchdog kết thúc một bài kiểm tra con sau 20 s và báo deadlock.

Từ số lần lấy lock của từng thread, bài kiểm tra in tỉ số max/min (1,00 là chia đều). Khi mỗi thread có một core, `mcs` chia gần như đều (1,00 ở 4 thread, 1,01 ở 10 thread), đúng tính chất FIFO của hàng đợi MCS [5]. Ở 20 thread, MCS chỉ lấy lock 15 221 lần trong 300 ms (so với 2 705 564 lần ở 4 thread) với tỉ số 311,40, vì lock được trao cho một thread không có core.

Trên `apple-m5`, cả 12 lock đạt mọi bài kiểm tra con, kể cả khi quá tải. Dưới TSan cả 12 lock đạt và không có cảnh báo.

5.3 Sense-reversing barrier

Ở episode e (lần thứ e cả nhóm đi qua barrier), mỗi thread ghi e vào ô của mình, gọi `wait`, rồi đọc mọi ô. Một ô nhỏ hơn e nghĩa là thread này qua sớm. Một ô lớn hơn $e+1$ nghĩa là thread khác đã qua episode kế tiếp mà không chờ. Các ô là atomic `relaxed`, nên điều kiện chỉ đúng nhờ thứ tự do barrier tạo ra. Phép kiểm handoff dùng xen kẽ hai barrier với các ô là biến thường. Dưới TSan, nó kiểm memory order của barrier. Các pha chạy 1, 4, 10 và 20 thread, tới 300 000 episode liên tiếp. Watchdog báo treo khi không thread nào tiến trong 10 s.

Bản schedule fuzzing chèn một độ trễ ngẫu nhiên vào mỗi điểm giữa hai bước của giao thức, để một thread bị chậm đúng giữa hai bước.

Trên `apple-m5`, cả 7 barrier đạt với tổng cộng 2 198 144 episode được kiểm. Bản fuzzing đạt cho 5 barrier tự viết. Pha 20 thread của `sr` và `sr_bo_spin` hết ngân sách 1,5 s sau 64 và 80 episode. Dưới TSan cả 7 barrier đạt. Nhóm bỏ phép kiểm handoff riêng cho `std_barrier`, vì libc++ của macOS không được instrument và TSan báo cảnh báo giả.

5.4 Concurrent counter

`increment()` trả về giá trị trước khi tăng. Nhóm lưu mọi giá trị trả về và kiểm rằng tập giá trị trả về đúng là $\{0, \dots, M-1\}$. Dãy của mỗi thread phải tăng ngặt. Ở một lượt riêng, nếu lần gọi A trả về trước khi B bắt đầu thì giá trị của A phải nhỏ hơn. Các phép kiểm này là bằng chứng linearizability [4]. Tập giá trị đúng là $\{0, \dots, M-1\}$ khi và chỉ khi có một thứ tự tuần tự hợp lệ. Hai phép kiểm thứ tự xác nhận thứ tự đó tôn trọng thời gian thực.

Ở `-O2`, compiler gộp một vòng tăng không atomic thành một lần đọc và một lần ghi, nên bài kiểm tra chỉ so giá trị cuối gần như luôn đạt với một counter không đồng bộ. Bài kiểm tra của nhóm lưu mọi giá trị trả về và cho địa chỉ counter đi qua `pc::opaque()` ở mỗi vòng, buộc mỗi lần tăng đọc và ghi thật.

Trên `apple-m5`, cả 19 counter đạt, tổng cộng hơn 121 triệu lần tăng được kiểm. Riêng `lock_mcs` hết ngân sách thời gian ở 20 thread và chỉ được kiểm qua 1 765 312 lần tăng. Dưới TSan cả 19 counter đạt, không có cảnh báo.

Biến thể	Lỗi cài vào	Stress (+ fuzzing)	TSan	Bị bắt bởi
`TAS_CHECK_THEN_SET`	`load` rồi `store` thay `exchange`	3/3 vi phạm loại trừ	race	cả hai
`TAS_RELEASE_RELAXED`	nhả lock bằng `store` relaxed	0/3	race	chỉ TSan
`MCS_NO_WAIT_SUCCESSOR`	nhả lock không chờ thread kế tiếp	3/3 treo	không	stress
`MCS_ALWAYS_CLEAR_TAIL`	luôn đặt `tail` về null	3/3 treo	race	cả hai
`single_flag`	một cờ, không đảo chiều	5/5 (5/5) sai episode	không	stress
`flip_before_reset`	đảo `sense` trước khi đặt lại bộ đếm	2/5 (5/5) treo	không	stress
`nonatomic_count`	`load`+`store` thay `fetch_add`	5/5 (5/5) treo	không	stress
`fetch_add_release_only`	`fetch_add` release thay acq_rel	1/5 (0/5) sai episode	race	chủ yếu TSan
`thread_local_sense`	local sense chung cho mọi barrier	5/5 (5/5) sai giá trị handoff	race	cả hai
`plain_long`	`long` thường	3/3 final, dup, miss, order	race	cả hai
`load_store`	`load` rồi `store` atomic	3/3 final, dup, miss, order	không	kiểm giá trị
`cas_no_retry`	CAS một lần, không thử lại	3/3 final, dup, miss	không	kiểm giá trị
`lock_read_outside`	đọc giá trị trước `acquire()`	3/3 final, dup, miss, order	race	cả hai
`block_reserve`	giữ trước 8 giá trị mỗi thread	3/3 chỉ real-time	không	kiểm giá trị

Bảng 3: Mutation testing trên `apple-m5`: lock (trên), barrier (giữa), counter (dưới). Cột stress: số lần chạy bắt được trên số lần chạy (trong ngoặc: bản schedule fuzzing) và dấu hiệu; với counter là các phép kiểm báo lỗi (final, duplicate, missing, order, real-time). Cột TSan: có báo data race không.

5.5 Mutation testing

Trả lời RQ1. Có. Trên `apple-m5`, mọi cài đặt (12 lock, 7 barrier, 19 counter) đạt mọi bài kiểm tra và TSan không báo data race nào. Bộ kiểm tra bắt cả 14/14 biến thể cố ý làm hỏng. Riêng hai lỗi memory order thì stress test chỉ bắt được 1 trong 8 lần chạy. TSan bắt được cả hai. Dự đoán D1.1 đúng. Kết luận dựa trên một lượt kiểm tra trên một máy ARM.

Mỗi biến thể sửa đúng một chỗ của một thuật toán đúng. Bảng 3 tóm tắt kết quả.

Hai biến thể chỉ TSan bắt ổn định đều là lỗi memory order. `TAS_RELEASE_RELAXED` đổi lệnh nhả lock `stlrb` (store release) thành `strb`. Ba lần chạy không có cập nhật nào bị mất, và trên x86-64 biến thể này sinh đúng lệnh của bản đúng. TSan bắt nó vì store relaxed không tạo cạnh happens-before giữa hai chủ lock liên tiếp. `fetch_add_release_only` đổi `ldaddal` thành `ldaddl`, nên thread tới cuối có thể đọc ô của thread khác trước khi `fetch_add` của nó có hiệu lực. TSan bắt nó qua các ô thường của phép kiểm handoff. Ngược lại, TSan im lặng với bảy biến thể chỉ dùng atomic hoặc gây treo. Không kỹ thuật nào một mình bắt được mọi biến thể.

Dự đoán D1.1 đúng ở cả ba ý. Stress test hoặc phép kiểm giá trị bắt mọi lỗi thuật toán. Hai lỗi memory order chỉ lộ 1 trong 8 lần chạy và cần TSan. Bốn lỗi treo (hai biến thể MCS, `flip_before_reset`, `nonatomic_count`) hiện ra qua watchdog. D1.1 không lường trước một điều. Fuzzing làm `flip_before_reset` treo sau 5 episode (bản thường cần 39 896 và 289 052 episode), nhưng làm lỗi thiếu acquire biến mất (0/5).

5.6 Giới hạn của phương pháp

Stress test và fuzzing chỉ xét các lịch chạy đã xảy ra. Chúng chỉ ra được là có lỗi, không chứng minh được là không có lỗi. Tính đúng vì vậy dựa trên lập luận ở mục Cài đặt. Trên x86-64 (memory model TSO) các lỗi memory order sinh cùng lệnh máy với bản đúng nên không bao giờ lộ ra lúc chạy. Với loại lỗi này, nhóm chỉ có TSan. TSan chỉ tìm data race, chỉ thấy các đường đi đã chạy và cần mã được instrument. Các baseline OpenMP và các lock, barrier chỉ có trên Linux chưa chạy dưới TSan. Bộ kiểm tra không kiểm tiến triển (lock-free, wait-free) ngoài trường hợp treo, và không kiểm `get()` trong lúc các thread đang tăng. Các biến thể đột biến do chính nhóm chọn, nên chúng chỉ chứng tỏ bộ kiểm tra nhạy với những lỗi đó.

6 Phương pháp đo

Mỗi lần chạy in đúng một dòng CSV gồm tham số, thời gian, cột `ok` (phép kiểm của lần chạy) và load average. Đồng hồ chạy từ lúc mọi thread đã được tạo và cổng xuất phát chung mở ra tới khi lần `join` cuối trả về. Việc tạo thread nằm ngoài vùng đo.

Trong khung của đề, compiler được phép xoá `is_prime(N)` vì kết quả bị bỏ đi. Nhóm giữ `is_prime` không inline, cho `N` đi qua `pc::opaque()` ở mỗi lời gọi, và kiểm tổng các kết quả ở cuối lần chạy (cột `ok`). Bảng 5 xác nhận phần việc không bị xoá. Ở $N = 1009$, thời gian một lời gọi t chỉ thay đổi từ 504 tới 527 ns khi `OUTSIDE_WORK` tăng từ 0 lên 15, và t tỉ lệ với N. Cả 12 152 lần chạy đều có `ok = 1`.

Phần lock quét 12 lock qua ba lượt. Một lượt dùng $N = 1009$ với `OUTSIDE_WORK` (OW) bằng 0, 1, 3, 7, 15. Hai lượt còn lại dùng $N = 11$ và $N = 10007$ với OW bằng 0 và 3. N phải là số nguyên tố để `is_prime` chạy đủ $N - 2$ vòng. Ba giá trị cho critical section khoảng 5 ns, 0,5 µs và 5 µs. OW quyết định mức tranh chấp. Theo mô hình, lock bão hoà khi $p > 1 + \mathrm{OW}$. Barrier quét 7 cài đặt. Counter quét 19 cài đặt với OW bằng 0, 1, 8 và $N = 101$. Số thread trên `apple-m5` là 1, 2, 3, 4, 6, 8, 10, 12, 16, 20. Trong đó 4 là số P-core, 10 là mọi core, 12–20 là quá tải.

Với lock, số vòng mỗi thread là $I = W_N / \max(p,\, 1 + \mathrm{OW})$, để cận dưới của mọi cấu hình khoảng 0,19 s. Nhóm hiệu chỉnh W_N một lần cho mỗi N bằng `ttas`. Với barrier và counter, chi phí một thao tác chênh nhau nhiều bậc, nên mỗi cấu hình có I riêng suy từ một lần chạy thử, nhắm 0,25 s và 0,15 s. I cố định cho cả đợt đo. Hình và bảng quy thời gian về cùng số vòng lặp.

Thời hạn mỗi lần chạy là 30 s cho lock và 10 s cho barrier và counter. Nhóm đo lock MCS quá tải với số vòng nhỏ hơn 100 lần ngay từ đầu (27 cấu hình). Lần chạy vượt thời hạn được ghi thành một dòng `ok=0`, không bỏ đi, rồi cấu hình đó chạy lại với số vòng nhỏ hơn 100 lần. Trên `apple-m5` có 9 lần như vậy, đều là MCS quá tải (6 ở lock, 3 ở counter).

Mỗi cấu hình chạy 7 lần. Hình báo cáo trung vị, thanh sai số đi từ min tới max. Nhóm không dùng trung bình vì phân bố lệch phải. Ở 33 % cấu hình lock, lần chậm nhất dài hơn 1,5 lần trung vị. Lần vượt thời hạn được tính là chậm nhất khi lấy trung vị. Trong mỗi lượt lặp, mọi cấu hình chạy theo thứ tự xáo trộn với seed cố định. Mỗi thời điểm chỉ có một tiến trình đo, và đợt đo chỉ bắt đầu khi load average không quá 7,0.

Phần lock được so với cận dưới $T_{\min} = \max(p\, I\, t,\ (1 + \mathrm{OW})\, I\, t)$. Vế đầu là tổng các critical section chạy nối tiếp, vế sau là phần việc của riêng một thread. t là trung vị lần chạy 1 thread của lock nhanh nhất chia cho $I(1 + \mathrm{OW})$. Thời gian đo chia cho $T_{\min}$ là hệ số chậm. Bảng 4 tóm tắt đợt đo. Bảng 5 cho t.

Bảng 4: Tóm tắt đợt đo trên máy `apple-m5`: số lần chạy, số lần vượt thời hạn và tải nền ghi trong từng dòng số liệu.

	Lock	Barrier	Counter	Lock (`yield`)
Số cài đặt	12	7	19	2
Số cấu hình	1080	70	570	16
Số lần lặp mỗi cấu hình	7	7	7	7
Số lần chạy đã ghi	7 566	490	3 993	112
Cấu hình đo với số vòng lặp giảm	27	0	–	0
Lần chạy vượt thời hạn	6	0	3	0
Lần chạy bỏ qua sau khi vượt thời hạn	0	0	0	0
Lần chạy hỏng phép kiểm (`ok=0`)	0	0	0	0
Load average 1 phút (nhỏ nhất – lớn nhất)	12,4 – 51,2	13,2 – 48,0	12,4 – 51,2	14,2 – 45,2
Bắt đầu (ngày/tháng giờ:phút)	14/10 10:13	14/10 10:13	14/10 10:13	14/10 10:17
Kết thúc	14/10 11:37	14/10 11:36	14/10 11:37	14/10 11:37

Bảng 5: Đơn vị công việc trên máy `apple-m5`: số vòng lặp I của lần chạy 1 thread, thời gian chạy 1 thread của lock nhanh nhất, và t = thời gian đó chia cho số lời gọi `is_prime`, $I\,(1 + \mathrm{OW})$. Nếu với cùng N mà t không đổi theo `OUTSIDE_WORK` thì thời gian tăng tuyến tính theo số lời gọi, tức phần việc không bị compiler xoá. t còn chứa chi phí lấy và nhả lock chia cho $1 + \mathrm{OW}$.

N	`OUTSIDE_WORK`	I (vòng lặp mỗi thread)	Thời gian 1 thread (s)	t (ns)
11	0	43 410 432	0,199	4,60
11	3	10 852 608	0,199	4,58
1009	0	343 663	0,175	511
1009	1	171 832	0,181	526
1009	3	85 916	0,173	504
1009	7	42 958	0,178	519
1009	15	21 479	0,181	527
10007	0	36 381	0,188	5 160
10007	3	9 095	0,189	5 190

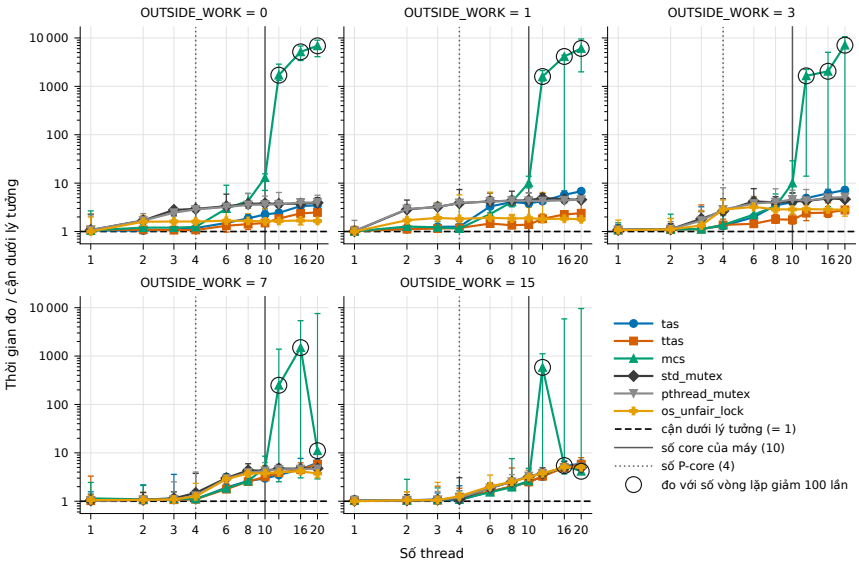
7 Kết quả và phân tích

7.1 Mutex lock

Độ chậm là thời gian đo chia cho cận dưới lý tưởng $\max(p,\, 1 + \mathrm{OW})\, I\, t$ (mục Phương pháp đo). Độ chậm 1 nghĩa là lock không thêm chi phí nào. Độ công bằng là thời gian của thread xong sớm nhất chia cho thời gian của thread xong muộn nhất trong một lần chạy. Mọi thread lấy lock cùng số lần, nên 1 là hoàn toàn đều.

7.1.1 RQ2: Lock mở rộng thế nào theo số thread, tranh chấp và độ dài critical section?

Trả lời RQ2. TTAS mở rộng tốt nhất trong ba lock tự viết. Ở 20 thread ($N = 1009$, $\mathrm{OW} = 0$), nó chậm 2,48 lần cận dưới. MCS chậm khoảng 6 900 lần vì sụp ngay khi số thread chạm số core. Lock thư viện không sụp. Chúng chậm hơn spinlock khi critical section dài cỡ 0,5 μs và nhanh hơn khi critical section chỉ vài nanosecond. Kết luận chỉ áp dụng cho `apple-m5`.



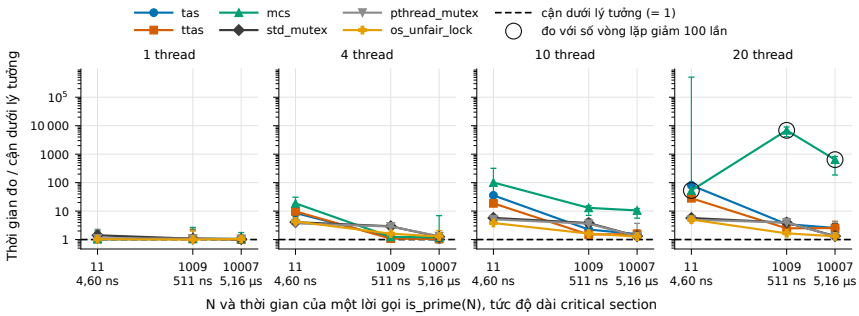
Hình 1: Thời gian đo chia cho cận dưới lý tưởng $\max(p\,I\,t,\ (1+\mathrm{OW})\,I\,t)$, $N = 1009$: giá trị 1 nghĩa là lock không thêm chi phí nào ngoài phần việc bắt buộc tuần tự. Điểm là trung vị của 7 lần lặp, thanh sai số đi từ giá trị nhỏ nhất tới giá trị lớn nhất. Vạch dọc liền: số core của máy (10); vạch chấm: số P-core (4). Điểm có vòng tròn bao quanh: cấu hình sụp đổ (hoặc đã vượt thời hạn ở số vòng lặp đầy đủ) nên được đo với số vòng lặp nhỏ hơn 100 lần; giá trị vẽ đã quy đổi tuyến tính về cùng số vòng lặp với các điểm khác.

Bằng chứng. Ở OW = 0, ba spinlock tự viết chậm không quá 1,24 lần cận dưới tới 4 thread (Hình 1). Mọi đường cong gãy từ 4 lên 6 thread, tức tại số P-core chứ không tại số core. MCS gãy mạnh nhất, chậm 13 lần ở đúng 10 thread rồi sụp hẳn khi vượt số core (Bảng 6). TAS và TTAS chỉ chậm dần. Lock thư viện gần như phẳng từ 8 thread.

Bảng 6: Trung vị thời gian chạy (s) của mọi lock trên máy `apple-m5`, $N = 1009$, `OUTSIDE_WORK` = 0, quy về $I = 343\,663$ vòng lặp mỗi thread (thời gian đo nhân với $I/I_{\mathrm{đo}}$). Ký hiệu $> x$: trung vị vượt thời hạn của một lần chạy. †: đo với số vòng lặp nhỏ hơn 100 lần (cấu hình sụp đổ hoặc đã vượt thời hạn), rồi quy đổi tuyến tính.

Lock \ số thread	1	2	3	4	6	8	10	12	16	20
`tas`	0,184	0,367	0,572	0,827	1,58	2,63	3,95	5,12	9,42	12,2
`tas_bo_sleep`	0,177	0,386	0,577	0,782	1,17	1,50	1,86	2,29	3,16	3,78
`tas_bo_spin`	0,189	0,373	0,554	0,774	1,35	2,24	2,75	3,62	6,27	9,11
`tas_bo_yield`	0,178	0,377	0,575	0,750	1,14	1,55	1,97	2,29	3,17	3,95
`ttas`	0,189	0,380	0,562	0,756	1,38	1,99	2,63	3,90	6,65	8,70
`ttas_bo_sleep`	0,192	0,360	0,561	0,764	1,27	1,89	2,64	3,43	6,04	9,99
`ttas_bo_spin`	0,196	0,364	0,574	0,777	1,33	1,98	2,90	3,85	6,01	9,31
`ttas_bo_yield`	0,203	0,372	0,573	0,750	1,26	1,86	2,64	3,39	5,94	7,75
`mcs`	0,185	0,422	0,633	0,872	3,13	6,04	22,8	3 590†	14 500†	24 100†
`std_mutex`	0,188	0,594	1,47	2,06	3,51	5,23	6,67	7,82	10,6	13,8
`pthread_mutex`	0,188	0,590	1,29	2,01	3,38	4,97	6,32	7,72	11,1	13,9
`os_unfair_lock`	0,175	0,558	0,848	1,13	1,76	2,32	2,84	3,48	4,74	5,81
cận dưới lý tưởng	0,175	0,351	0,526	0,702	1,05	1,40	1,75	2,11	2,81	3,51

Ô OW = 15 là một thí nghiệm đối chứng, vì với $p \le 15$ lock bận tối đa $p/16$ thời gian. Vậy mà ở 6 thread cả mười hai lock, kể cả lock thư viện, cùng chậm 1,53–2,03 lần cận dưới.



Hình 2: Ảnh hưởng của độ dài critical section: thời gian đo chia cho cận dưới lý tưởng theo N, `OUTSIDE_WORK` = 0, mỗi ô một số thread. Dưới mỗi N là thời gian của một lời gọi `is_prime(N)`. Điểm là trung vị của 7 lần lặp, thanh sai số đi từ giá trị nhỏ nhất tới giá trị lớn nhất. Điểm có vòng tròn bao quanh: cấu hình sụp đổ (hoặc đã vượt thời hạn ở số vòng lặp đầy đủ) nên được đo với số vòng lặp nhỏ hơn 100 lần; giá trị vẽ đã quy đổi tuyến tính về cùng số vòng lặp với các điểm khác.

Với $N = 10\,007$ (khoảng 5,2 μs) TAS và TTAS gần như trùng nhau (Hình 2). Với $N = 11$ (4,6 ns), ở 20 thread TAS chậm 80,5 lần cận dưới, TTAS 28,2 lần, còn lock thư viện chỉ khoảng 5 lần.

Vì sao. Chỗ gãy ở 4 thread đến từ máy, không từ thuật toán. Ở ô đối chứng, mọi lock cùng chậm như nhau. Độ công bằng ở 6 thread là 0,84–0,98, tức mọi thread cùng chậm đi. Điều này phù hợp với việc scheduler luân chuyển thread giữa P-core và E-core, hoặc với tải nền. Nhóm không tách được hai khả năng.

Nhóm giải thích sự sụp đổ của MCS ở 10 thread bằng một giả thuyết. MCS trao lock cho đúng thread kế tiếp theo thứ tự FIFO (vào trước được phục vụ trước). Mỗi lần thread đó không có core, cả hàng mất trắng một lượt lập lịch. TAS và TTAS không bị như vậy, vì thread nào đang chạy cũng lấy được lock rảnh. Ba số đo ủng hộ giả thuyết. Độ công bằng của MCS bằng 1 tới 10 thread (Hình 4), tức lock luôn đi đúng thứ tự hàng đợi. Cột `load1` của bảy lần chạy là 21,8–33,7, lớn hơn số core, nên với tải nền máy đã quá tải ngay ở 10 thread. Cuối cùng, một critical section của MCS ở 10 thread tốn trung bình 6,6 μs. Nếu mỗi lượt mất trắng tốn cỡ 0,87 ms như đo được ở 12 thread, chỉ cần khoảng 0,7 % số lần chuyển giao rơi vào thread không có core là đủ giải thích. Đếm số lần chuyển ngữ cảnh không tự nguyện của từng thread (`getrusage`) trong lần chạy sẽ xác nhận hoặc bác bỏ giả thuyết này.

Lock thư viện cho thread chờ ngủ trong kernel, nên không giành core của chủ lock và không sụp. Việc chúng chậm ở $N = 1009$ mà nhanh ở $N = 11$ có một giả thuyết. Mỗi lần đánh thức một thread qua kernel tốn một chi phí cố định cỡ 1,5 μs. Trong lúc thread được đánh thức chưa kịp chạy, thread vừa nhả lock lấy lại lock và chạy tiếp các critical section của mình với cache line còn nóng. Ở $N = 1009$, mỗi critical section của `std_mutex` tốn thêm 1 459 ns so với khi không tranh chấp, tức gần như mỗi lần chuyển giao trả trọn một lần đánh thức.

Ở $N = 11$, cùng lock tốn thêm 19,7 ns mỗi critical section, tức một lần đánh thức được chia cho khoảng 74 critical section. Chuỗi này kéo dài khoảng 1,9 μs, cùng bậc với lần ngủ ngắn nhất của `sleep_for` (2 500 ns). Số đo này nhỏ hơn một lần chuyển cache line của TAS (39,5 ns ở 4 thread), nên phần lớn critical section không thể có chuyển giao. Độ công bằng 0,92 của `std_mutex` ở 20 thread không chống lại giả thuyết, vì thước đo này không thấy các đợt độc chiếm luân phiên. Đếm số lời gọi hệ thống đánh thức trên mỗi critical section ở hai giá trị N sẽ kiểm tra được giả thuyết.

TAS thua TTAS nhiều nhất khi lock đổi chủ ở mỗi lượt. Phép nhả lock của TAS phải tranh cache line (đơn vị 128 byte phần cứng giữ nhất quán giữa các core) với các `exchange` liên tục của thread chờ, còn thread TTAS chờ bằng phép đọc. Độ công bằng của TAS 10 thread là 0,32 ở OW = 0 và 0,91 ở OW = 3. Khi không có việc ngoài, lock chủ yếu ở lại với thread vừa nhả.

Độ dài critical section quyết định tỉ lệ giữa chi phí chuyển giao và phần việc (Hình 2). Ở $N = 11$

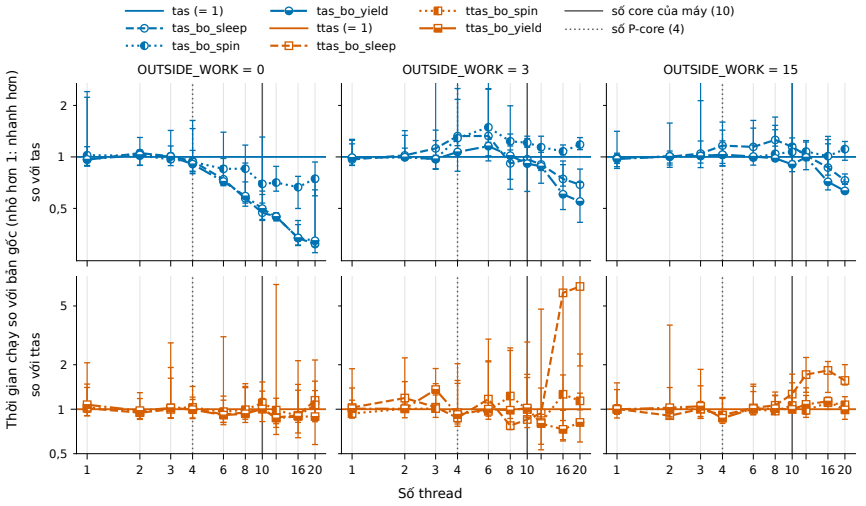
và 4 thread, một critical section của TAS tốn 39,5 ns, gấp khoảng 8,6 lần lời gọi `is_prime(11)`. Ở $N = 10\,007$, cùng chi phí đó chưa tới 1 % của 5,2 μs, nên mọi lock gần như trùng nhau.

Dự đoán. D2.1 đúng một nửa. TAS chậm hơn TTAS khi nhiều thread chờ. Nhưng MCS không nhanh nhất khi chưa vượt số core, mà chậm nhất từ 6 thread và sụp ngay tại số core. D2.2 đúng cho TAS và TTAS. D2.3 chỉ đúng khi $1 + \mathrm{OW} \leq 4$. Xa hơn, máy giới hạn trước lock. D2.4 đúng, và MCS sụp sớm hơn dự kiến.

Độ vững. Cả bảy lần chạy MCS 10 thread đều chậm 7,1–15,5 lần cận dưới. Các điểm MCS vượt số core ở $N = 11$ không đo được sự sụp đổ. Lần chạy đầu vượt thời hạn 30 s, còn các lần sau giảm vòng lặp tới mức hàng đợi không kịp hình thành. Với $\mathrm{OW} \geq 7$, MCS vượt số core cho kết quả hai cực, nên chỉ khoảng min–max đáng tin.

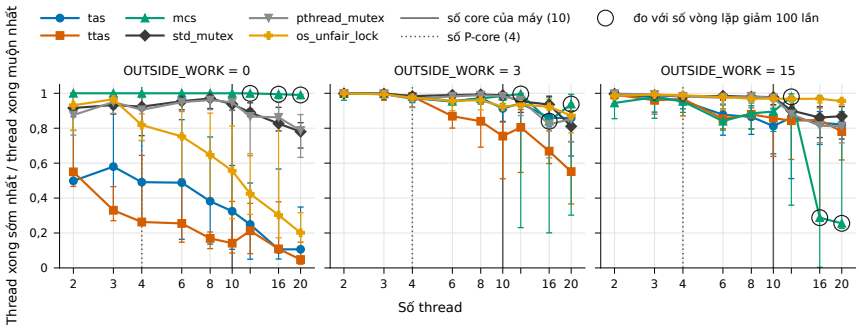
7.1.2 RQ3: Backoff giúp hay hại, và lock nhanh nhất có công bằng không?

Trả lời RQ3. Backoff giúp TAS và gần như không đổi gì với TTAS. Ở 20 thread ($N = 1009$, $\mathrm{OW} = 0$), backoff bằng `sleep_for` đưa TAS từ 12,2 s xuống 3,78 s. Lợi ích đến từ việc lock gần như không đổi chủ, nên lock nhanh nhất lại kém công bằng nhất. Khi có việc ngoài critical section và tranh chấp vừa phải, backoff làm chậm. Kết luận chỉ áp dụng cho `apple-m5`.



Hình 3: Tác dụng của backoff, $N = 1009$ (critical section khoảng 511 ns): thời gian chạy của từng biến thể có backoff chia cho trung vị của lock gốc không backoff (đường ngang ở 1). Hàng trên: họ TAS; hàng dưới: họ TTAS. Kiểu đường và kiểu tô của điểm cho biết chính sách chờ: ngủ bằng `sleep_for` như mẫu của đề (`_bo_sleep`), quay trong user space (`_bo_spin`), quay rồi nhường core (`_bo_yield`). Điểm là trung vị của 7 lần lặp, thanh sai số đi từ giá trị nhỏ nhất tới giá trị lớn nhất. Vạch dọc liền: số core của máy (10); vạch chấm: số P-core (4).

Bằng chứng. Ở $\mathrm{OW} = 0$ (Hình 3, cột trái), `tas_bo_sleep` và `tas_bo_yield` chỉ mất khoảng 0,31 lần thời gian của TAS ở 20 thread, còn các biến thể của TTAS nằm trong 0,87–1,15 lần TTAS. Ở $\mathrm{OW} = 3$ và 4–6 thread, backoff làm TAS chậm hơn. Ở 20 thread, TAS có backoff lại nhanh hơn.



Hình 4: Độ công bằng, $N = 1009$. Mọi thread lấy lock cùng số lần, nên lock không công bằng lộ ra qua thời gian: có thread chạy xong vòng lặp của mình sớm hơn nhiều so với thread khác. Trục tung là thời gian của thread xong sớm nhất chia cho thời gian của thread xong muộn nhất trong cùng một lần chạy; 1 là hoàn toàn đều. Điểm là trung vị của 7 lần lặp, thanh sai số đi từ giá trị nhỏ nhất tới giá trị lớn nhất. Vạch dọc liền: số core của máy (10); vạch chấm: số P-core (4). Cấu hình có trung vị vượt thời hạn không được vẽ. Điểm có vòng tròn bao quanh: cấu hình sụp đổ (hoặc đã vượt thời hạn ở số vòng lặp đầy đủ) nên được đo với số vòng lặp nhỏ hơn 100 lần; giá trị vẽ đã quy đổi tuyến tính về cùng số vòng lặp với các điểm khác.

Ở OW = 0, MCS giữ độ công bằng 1 tới 10 thread (Hình 4). Độ công bằng của `tas_bo_sleep` bằng đúng $1/p$, tới 0,05 ở 20 thread, tức là các thread chạy xong lần lượt từng thread một. TAS và TTAS cũng thiên vị thread vừa nhả lock, và `os_unfair_lock`, lock thư viện nhanh nhất, kém công bằng nhất trong ba lock thư viện.

Vì sao. Các biến thể `tas_bo_*` lùi lại sau mọi lần `exchange` thất bại, kể cả khi lock chỉ đang bị giữ, nên thread chờ gần như luôn ngủ hoặc nhường core. Các biến thể `ttas_bo_*` chỉ lùi lại sau một lần `exchange` thua và vẫn đọc liên tục như TTAS (mục Cài đặt). Một lần `sleep_for` yêu cầu 100 ns thật ra ngủ 2 670 ns, đủ để chủ lock chạy khoảng năm critical section 511 ns liền nhau. Độ công bằng $1/p$ là bằng chứng trực tiếp.

Khi OW > 0, lock rảnh trong lúc thread chờ còn ngủ ít nhất 2,5 μs. Ở tranh chấp vừa phải, đó là thời gian mất, còn ở số thread lớn luôn có thread sẵn sàng. Khi vượt số core, thread MCS bị lấy core giữa `release` và `acquire` không nằm trong hàng đợi, nên ở OW = 15 và 20 thread MCS có độ công bằng thấp nhất.

Khi vượt số core và OW $\geq$ 3, `ttas_bo_sleep` chậm khoảng 6 lần TTAS. Nhóm giả thuyết rằng ở biến thể này một phần thread chờ quay đọc và chiếm core, phần khác ngủ. Thread vừa thức dậy sau khi ngủ có thể giành core của chủ lock. Trong lúc chủ lock không chạy, các thread quay đọc đốt hết lượt của mình. Hai số đo ủng hộ giả thuyết này. `ttas_bo_yield`, chỉ khác ở chỗ nhường core thay vì ngủ, không chậm hơn TTAS (1,97 s so với 2,42 s ở 20 thread, OW = 3). `tas_bo_sleep`, nơi mọi thread chờ đều ngủ, cũng không bị. Đếm số lần chủ lock bị lấy core trong lúc giữ lock sẽ kiểm tra giả thuyết này.

Dự đoán. D3.1 đúng. D3.2 đúng khi OW = 0. Khi OW > 0, nó chỉ đúng ở tranh chấp vừa phải, sai ở số thread lớn. D3.3 đúng khi mọi thread được chạy. Khi vượt số core, MCS lại kém công bằng nhất.

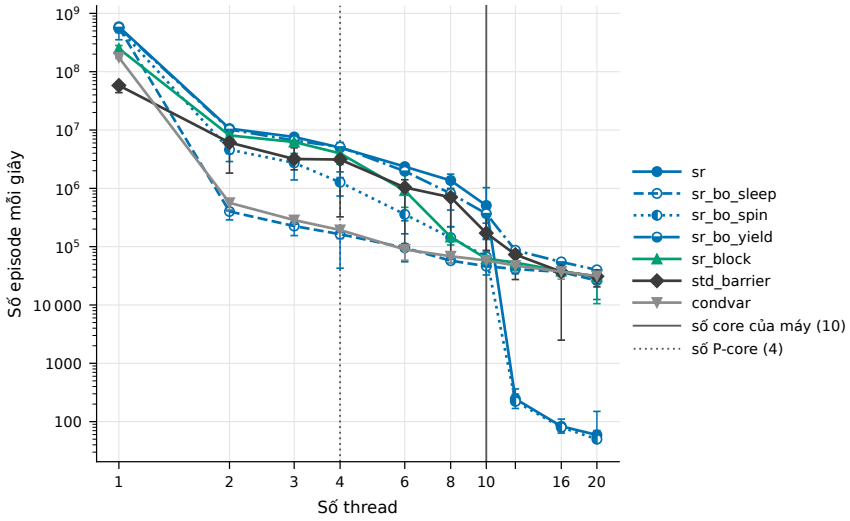
Độ vững. Kết quả của `ttas_bo_sleep` khi vượt số core phân tán mạnh giữa các lần lặp. Đổi lệnh chờ `isb` thành `yield` không tạo khác biệt vượt độ phân tán.

7.2 Barrier

7.2.1 RQ4: Chi phí một barrier episode

Trả lời RQ4. Với chờ spin, một episode tốn thêm khoảng 53 ns cho mỗi thread trong cụm 4 P-core, dốc gần gấp 6 khi thread tràn sang E-core, và sụp khoảng 2 000 lần khi số thread vượt số core. Khi mỗi thread có core riêng, spin thuần nhanh nhất. Khi vượt số core, `sr_bo_yield` nhanh nhất. Kết luận chỉ áp cho `apple-m5`.

Nhóm đo số episode (một lần cả p thread cùng qua barrier) mỗi giây khi giữa hai barrier không có việc gì (Hình 5, Bảng 7).



Hình 5: Số barrier episode hoàn thành mỗi giây theo số thread; giữa hai barrier không có việc gì, nên đây là chi phí thuần của việc đồng bộ. Trục tung theo thang logarit. `sr`: sense-reversing barrier chờ bằng vòng spin; `sr_bo_*`: thêm backoff; `sr_block`: chờ chặn. Số episode của mỗi cấu hình được hiệu chỉnh riêng để mọi lần chạy dài cỡ như nhau. Điểm là trung vị của 7 lần lặp, thanh sai số đi từ giá trị nhỏ nhất tới giá trị lớn nhất. Vạch dọc liền: số core của máy (10); vạch chấm: số P-core (4).

Bảng 7: Trung vị số barrier episode mỗi giây trên máy `apple-m5` theo số thread (k = nghìn, M = triệu; t/h = vượt thời hạn).

Barrier \ số thread	1	2	3	4	6	8	10	12	16	20
`sr`	589 M	10,6 M	7,60 M	5,00 M	2,36 M	1,36 M	508 k	246	83,2	59,1
`sr_bo_sleep`	572 M	401 k	226 k	164 k	95,6 k	57,5 k	46,3 k	41,3 k	36,8 k	26,2 k
`sr_bo_spin`	550 M	4,61 M	2,73 M	1,27 M	358 k	143 k	67,2 k	222	79,8	50,0
`sr_bo_yield`	573 M	10,2 M	6,61 M	5,12 M	1,96 M	817 k	368 k	86,5 k	54,7 k	40,0 k
`sr_block`	250 M	8,09 M	6,22 M	3,98 M	913 k	147 k	62,3 k	53,3 k	38,2 k	30,4 k
`std_barrier`	58,1 M	6,10 M	3,19 M	3,12 M	1,04 M	703 k	171 k	73,2 k	37,1 k	31,0 k
`condvar`	176 M	565 k	286 k	193 k	89,5 k	68,4 k	58,1 k	47,5 k	37,3 k	31,4 k

Với `sr`, mỗi thread thêm vào tốn khoảng 53 ns từ 2 tới 4 thread và khoảng 295 ns từ 4 tới 10 thread. Phần tuyến tính đến từ p phép `fetch_add` lần lượt trên cache line của bộ đếm và $p-1$ lần đọc lại cờ sau khi cờ đảo. Chỗ gãy sau 4 thread trùng với số P-core, và kết quả lock gãy ở cùng chỗ ngay cả ở ô đối chứng mà lock gần như rảnh (RQ2). Vì vậy nó đến từ máy chứ không từ thuật toán barrier. Nhóm giả thuyết hai nguyên nhân là E-core chậm hơn và cache line phải đi giữa hai cụm L2. Vì không gắn được thread vào core, nhóm không tách được hai nguyên nhân này.

Từ 10 lên 12 thread, `sr` chậm đi khoảng 2 070 lần, và mỗi thread vượt số core thêm khoảng 2 ms vào một episode. Một episode cần *mọi* thread được chạy, nhưng thread đã tới quay giữ core tới hết lượt lập lịch. Barrier giống MCS ở điểm này (RQ2).

Chờ chặn hoà vốn đúng tại số core. Dưới đó, `sr_block` luôn chậm hơn `sr`, vì đánh thức qua kernel tốn vài micro giây. Ở 12 thread, nó nhanh hơn khoảng 217 lần. `sr_bo_yield` nhanh hơn `std_barrier` ở mọi số thread từ 2 tới 20, và nhanh nhất trong mọi barrier khi vượt số core (11,6 μs mỗi episode ở 12 thread). Thread chờ gọi `sched_yield`, nhường core cho đúng thread chưa tới barrier. Nhờ vậy episode xong mà không phải chờ hết lượt lập lịch, và không phải trả một lần đánh thức qua kernel như `sr_block`. Khi mỗi thread có core riêng, lời gọi hệ thống này chỉ làm thread chậm nhận ra cờ đã đảo, nên `sr_bo_yield` chậm hơn `sr` tới 1,7 lần (8 thread).

Backoff làm barrier chậm đi khi mỗi thread có core riêng (`sr_bo_spin` chậm hơn `sr` 2,3 tới 9,5 lần), vì thread chờ chỉ đọc cờ nên không có tranh chấp để bớt. Một episode của `sr_bo_sleep` ở 2 thread tốn 2,49 μs, xấp xỉ thời gian ngủ thật ngắn nhất của `sleep_for` khi yêu cầu 100 ns (2,54 μs).

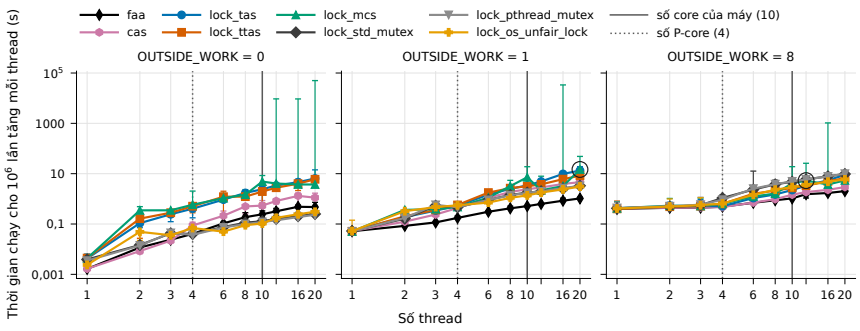
Đối chiếu với dự đoán, D4.1 chỉ đúng từng đoạn, vì chi phí tăng tuyến tính theo p nhưng độ dốc đổi ở 4 thread. D4.2 đúng. Spin sụp ngay khi vượt số core, và chờ chặn chỉ thắng ở vùng đó. D4.3 đúng. Backoff làm barrier chậm khi mỗi thread có core riêng. Các ô sụp đổ dựa trên 50–69 episode mỗi lần chạy nên chỉ cho bậc độ lớn. Khoảng cách 2–3 bậc vẫn lớn hơn nhiều độ phân tán giữa 7 lần lặp.

7.3 Concurrent counter

7.3.1 RQ5: Thứ tự hiệu năng của các counter

Trả lời RQ5. Khi không có việc giữa hai lần tăng, fetch-and-add nhanh hơn CAS 2,1 tới 2,7 lần từ 4 thread, vì CAS hỏng và phải thử lại. Counter dùng spinlock không backoff chậm nhất. Mutex thư viện và CAS có backoff lại nhanh hơn fetch-and-add ở chế độ này vì chúng dồn các lần tăng về một core. Lợi thế này mất khi có việc giữa hai lần tăng. Kết luận chỉ áp cho `apple-m5`.

Nhóm đo thời gian để mỗi thread tăng counter 10^6 lần, với OW lời gọi `is_prime(101)` giữa hai lần tăng (Hình 6, Bảng 8), cùng thời gian trung bình một lần tăng và độ công bằng như ở RQ3.



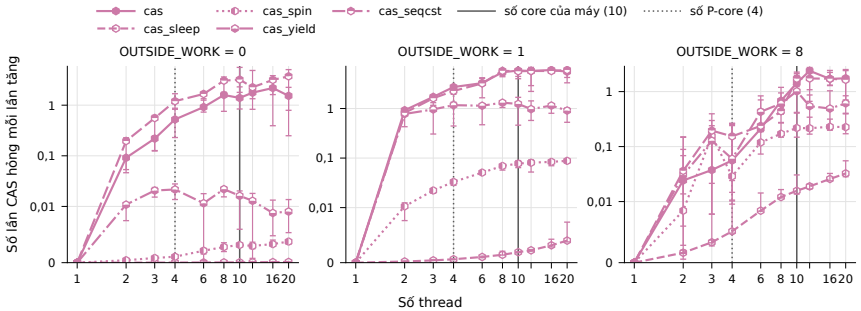
Hình 6: Thời gian chạy của các counter theo số thread p, mỗi ô một lượng việc giữa hai lần tăng (`OUTSIDE_WORK` lời gọi `is_prime(101)`). Mỗi thread tăng counter I lần; vì một lần tăng tốn từ vài nanosecond tới hàng trăm microsecond tuỳ cấu hình, I được hiệu chỉnh riêng cho từng cấu hình để mỗi lần chạy dài cỡ như nhau, và hình vẽ thời gian đo nhân với $10^6/I$, tức thời gian chạy nếu mỗi thread tăng 10^6 lần. `faa`: fetch-and-add; `cas`: vòng lặp compare-and-swap; `lock_X`: counter bảo vệ bằng lock `X` của phần 1. Các biến thể có backoff và `seq_cst` nằm ở hình riêng. Điểm là trung vị của 7 lần lặp, thanh sai số đi từ giá trị nhỏ nhất tới giá trị lớn nhất. Vạch dọc liền: số core của máy (10); vạch chấm: số P-core (4).

Không counter nào mở rộng. Với OW = 0, thông lượng của `faa` giảm từ khoảng 610 triệu lần tăng mỗi giây với 1 thread xuống 42 triệu với 20 thread, vì mỗi lần tăng phải giữ cache line của counter ở trạng thái độc quyền.

Bảng 8: Trung vị thời gian chạy (s) của mọi counter trên máy `apple-m5`, `OUTSIDE_WORK` = 0 (lời gọi `is_prime(101)` giữa hai lần tăng), quy về 10^6 lần tăng mỗi thread (thời gian đo nhân với $10^6/I$; I được hiệu chỉnh riêng cho từng cấu hình). Ký hiệu $> x$: trung vị vượt thời hạn của một lần chạy.

Counter \ số thread	1	2	3	4	6	8	10	12	16	20
`faa`	0,00164	0,0117	0,0238	0,0406	0,106	0,187	0,243	0,321	0,483	0,472
`faa_seqcst`	0,00166	0,0125	0,0219	0,0389	0,0911	0,163	0,299	0,343	0,448	0,557
`cas`	0,00167	0,00842	0,0219	0,0871	0,213	0,498	0,540	0,823	1,31	1,12
`cas_sleep`	0,00165	0,00341	0,00513	0,00695	0,0103	0,0138	0,0173	0,0211	0,0280	0,0351
`cas_spin`	0,00192	0,00385	0,00597	0,00772	0,0141	0,0216	0,0213	0,0271	0,0345	0,0461
`cas_yield`	0,00165	0,00340	0,00512	0,00706	0,0109	0,0162	0,0196	0,0220	0,0301	0,0360
`cas_seqcst`	0,00170	0,0177	0,0546	0,159	0,405	0,942	1,25	1,06	2,04	2,91
`lock_tas`	0,00426	0,108	0,244	0,407	0,931	1,71	2,41	3,38	4,59	6,45
`lock_tas_bo_sleep`	0,00421	0,00797	0,0121	0,0164	0,0246	0,0329	0,0456	0,0490	0,0681	0,0856
`lock_tas_bo_spin`	0,00428	0,00805	0,0122	0,0169	0,0450	0,0819	0,128	0,166	0,197	0,423
`lock_tas_bo_yield`	0,00408	0,0671	0,169	0,272	0,567	1,21	1,37	2,25	2,63	3,13
`lock_ttas`	0,00443	0,163	0,281	0,509	1,22	1,22	1,95	2,77	4,01	5,99
`lock_ttas_bo_sleep`	0,00408	0,00816	0,0122	0,0169	0,0247	0,0329	0,0407	0,0490	0,0665	0,0878
`lock_ttas_bo_spin`	0,00408	0,00802	0,0128	0,0179	0,0322	0,0567	0,0778	0,0983	0,149	0,189
`lock_ttas_bo_yield`	0,00399	0,0233	0,0955	0,0656	0,123	0,118	0,289	0,178	0,288	0,473
`lock_mcs`	0,00437	0,349	0,351	0,584	1,06	1,47	4,85	4,07	3,73	3,71
`lock_std_mutex`	0,00389	0,0146	0,0428	0,0402	0,0757	0,113	0,129	0,160	0,205	0,250
`lock_pthread_mutex`	0,00351	0,0140	0,0444	0,0371	0,0728	0,0993	0,120	0,140	0,189	0,233
`lock_os_unfair_lock`	0,00237	0,0488	0,0359	0,0693	0,0497	0,0898	0,105	0,176	0,245	0,306

`cas` chậm hơn `faa` 2,1 tới 2,7 lần từ 4 thread, và số lần CAS hỏng trên mỗi lần tăng vượt 1 từ 8 thread (Hình 7).



Hình 7: Số lần compare-and-swap hỏng trên mỗi lần tăng thành công của các counter dùng CAS: bằng chứng trực tiếp về mức tranh chấp. Trục tung tuyến tính dưới 0,01 và logarit phía trên; 0 nghĩa là không lần thử nào hỏng. Điểm là trung vị của 7 lần lặp, thanh sai số đi từ giá trị nhỏ nhất tới giá trị lớn nhất. Vạch dọc liền: số core của máy (10); vạch chấm: số P-core (4).

Một lần CAS hỏng đã trả giá một lần kéo cache line về mà không tiến được, còn `ldadd` của fetch-and-add đọc và ghi trong một bước. Nếu mỗi lần hỏng tốn một lần chuyển, thời gian của `cas` xấp xỉ thời gian của `faa` nhân $1 + f$, với f là số lần hỏng mỗi lần tăng. Từ 6 tới 20 thread, mô hình lệch tỉ số đo được không quá 15%. Ở 2 thread, ngược với D5.1, `cas` nhanh hơn `faa` khoảng 28%. Ở đó chỉ có 0,09 lần CAS hỏng trên mỗi lần tăng, nên gần như mọi lần tăng thành công ngay lần đầu, và mô hình $1 + f$ không giải thích được phần chênh còn lại. Kiến trúc ARMv8.1 cho phép phần cứng thực hiện `ldadd` ở gần cache dùng chung thay vì trong L1 của core gọi. Nếu M5 làm vậy, mỗi lần `ldadd` tốn một lượt đi về tới cache dùng chung. Một thread chạy vòng CAS thì có thể tăng vài lần liên tiếp trên line đang nằm sẵn trong L1 của nó. Đây là giả thuyết. Ghi

lại giá trị trả về của từng lần tăng sẽ cho biết mỗi thread tăng liên tiếp bao nhiêu lần trước khi line rời core của nó, và kiểm được giả thuyết này.

Khi OW ≤ 1, counter dùng spinlock không backoff chậm nhất. Ở 20 thread, `lock_tas` chậm hơn `faa` 13,7 lần. `lock_mcs` chậm hơn `lock_tas` 3,2 lần ngay ở 2 thread, vì MCS trao lock theo thứ tự đến nên mỗi lần tăng chuyển lock và giá trị sang core khác.

Ngược với D5.1, ở OW $= 0$ counter dùng `lock_pthread_mutex` và `lock_std_mutex` nhanh hơn `faa` từ 4 thread. Ở 8 thread, một lần tăng tốn 12,4 và 14,1 ns, so với 23,4 ns của `faa`. Với `faa`, mọi thread tiến đều nhau nên gần như mỗi lần tăng phải kéo line của counter từ core khác về. Một lần tăng dưới mutex rẻ hơn thế, nên tính trung bình nó tốn ít hơn một lần chuyển line. Người giữ lock thực hiện nhiều lần tăng liên tiếp trên line đã nằm trong cache của mình, trong khi các thread chờ ngủ trong kernel và không tranh line. Độ công bằng theo thời điểm kết thúc của mutex vẫn khoảng 0,98, nên các chuỗi lần tăng liên tiếp này ngắn so với cả lần chạy. Khi có việc giữa hai lần tăng (OW $= 8$, 20 thread), mỗi lần lấy lock gần như luôn phải đánh thức một thread đang ngủ, và `lock_pthread_mutex` chậm hơn `faa` 4,9 lần.

Backoff dồn các lần tăng về một thread tại mỗi thời điểm. Thời gian một lần tăng của `cas_sleep` bằng giá trị 1 thread trong khoảng 11% ở mọi số thread và mọi OW, và độ công bằng của nó xấp xỉ $1/p$, đúng như khi các thread chạy lần lượt. Vì vậy ở 20 thread `cas_sleep` nhanh hơn `cas` 32 lần khi OW $= 0$ nhưng chậm hơn 3,1 lần khi OW $= 8$, lúc chạy tuần tự bỏ phí phần `is_prime` lẽ ra chạy song song.

Thứ tự `seq_cst` không làm chậm fetch-and-add, nhưng làm `cas` chậm khoảng 2 lần ở OW $= 0$ và gần gấp đôi số lần hỏng. Ở OW ≥ 1, khác biệt biến mất. Nhóm giả thuyết `ldar` và `casal` ngăn các lần lặp chồng lên nhau.

Đối chiếu với dự đoán, D5.1 đúng với `faa`, `cas` và spinlock từ 4 thread, nhưng sai ở hai điểm đã nêu (mutex thư viện ở OW $= 0$, và `cas` ở 2 thread). D5.2 đúng. Số lần CAS hỏng trên mỗi lần tăng tăng từ 0,09 ở 2 thread lên 1,59 ở 8 thread. D5.3 đúng và mạnh hơn dự đoán. Backoff đưa số lần hỏng về gần 0, nhưng bằng cách cho các thread chạy lần lượt, nên mất công bằng. D5.4 sai với CAS, vì `seq_cst` làm CAS chậm khoảng 2 lần khi không có việc giữa hai lần tăng. Nhóm không phân tích `lock_mcs` trên 10 thread, vì các ô đó dựa trên 20–200 lần tăng mỗi thread và có 3 lần vượt thời hạn (Bảng 4).

7.4 So sánh giữa các máy

7.4.1 RQ6: Kết luận trên máy khác

Trả lời RQ6. Chưa kiểm được, vì báo cáo mới có số đo của `apple-m5`. Hình so sánh tự xuất hiện khi có từ hai máy (Bảng 2).

Một máy x86 kiểm được ba giả thuyết. Với TSO, mọi phép đọc-sửa-ghi atomic là lệnh có tiền tố `lock`, nên `cas_seqcst` phải bằng `cas`. Nếu không, giải thích ở RQ5 sai. Với SMT, chi phí barrier và counter phải gãy ở số core vật lý, còn spin barrier chỉ sụp khi vượt số luồng phần cứng. Độ trễ của `pause` và thời gian ngủ thật trên Linux đổi chi phí của các biến thể backoff.

[Cần viết: ghi kết quả từng giả thuyết trên máy thứ hai và các khác biệt lớn nhất giữa các máy.]

8 Các yếu tố đe dọa độ tin cậy

Construct validity. Cận dưới của phần lock dùng t đo ở 1 thread, nên chứa chi phí lấy và nhả lock của lock nhanh nhất. Ở $N = 11$, chi phí này chênh 41 % giữa các lock. Cận dưới còn bỏ qua giới hạn 10 core khi $p > 10$ và OW > 0. Ở 20 thread và OW $= 15$, sàn thật cao hơn cận dưới vẽ trên hình 1,6 lần. Phép quy về cùng số vòng lặp giả định thời gian tỉ lệ với số vòng. Giả định này yếu

nhất ở MCS quá tải (27–45 vòng mỗi thread) và ở các ô barrier sụp đổ (50–69 episode mỗi lần chạy). Ở các ô có lần vượt thời hạn, trung vị tính cả lần đó ở dạng cận dưới, nên khác trung vị chỉ trên các lần chạy thành công. Ở OW = 8, thời gian của mọi counter, kể cả `faa`, tăng ít nhất gấp đôi từ 4 lên 10 thread. Nhóm không có phép đo đối chứng không counter để tách phần do E-core.

Internal validity. macOS không cho ghim thread, và `apple-m5` có 4 P-core và 6 E-core. Ở $N = 1009$, thời gian một lời gọi trong 420 lần chạy 1 thread đi từ 468 tới 1 713 ns. Nhóm cho rằng độ phân tán này phần lớn do thread rơi vào E-core, nhưng không tách được ảnh hưởng của E-core với tải nền. Load average ghi trong các dòng số liệu của phần lock nằm trong 12,4–51,2, cao hơn số thread của chính lần chạy. Phần lớn có thể do load average còn mang các lần chạy 20 thread trước đó. Nhóm không tách được phần do tiến trình nền. Nhóm không đọc nhiệt độ hay tần số CPU. Thời gian 1 thread không trôi quá 3,4 % qua 7 lượt lặp. Với 7 lần lặp, nhóm không tính khoảng tin cậy. Trung vị che phân bố hai đỉnh và đuôi chậm, chỉ thanh min–max cho thấy phần này.

External validity. Mọi số đo đến từ một máy ARM, một compiler (Apple clang 21, `-O2`) và một hệ điều hành. Trên x86, với g++ hay với scheduler khác, thứ hạng có thể đổi. `is_prime` không chạm dữ liệu dùng chung, trong khi critical section thật thường ghi dữ liệu dùng chung, nên chi phí chuyển giao lock thực tế có thể lớn hơn.

9 Kết luận

Nhóm cài đặt ba spinlock, một sense-reversing barrier và ba họ concurrent counter, kiểm tra chúng bằng stress test, ThreadSanitizer và mutation testing, rồi đo chúng trên `apple-m5`. Câu trả lời cho sáu câu hỏi nghiên cứu như sau.

RQ1. Mọi cài đặt qua mọi bài kiểm tra, và bộ kiểm tra bắt được cả 14 biến thể cố ý làm hỏng. Không kỹ thuật nào tự nó đủ. Lỗi thuật toán lộ ra dưới stress test, còn lỗi memory order như nhả lock bằng store `relaxed` chỉ ThreadSanitizer bắt được.

RQ2. Tới 4 thread, cả ba spinlock bám cận dưới lý tưởng trong khoảng 1,24 lần. Ở 20 thread, TTAS chậm 2,48 lần cận dưới và TAS 3,48 lần, đúng thứ tự mà lưu lượng coherence dự đoán. MCS sụp ngay khi số thread bằng số core, vì nó trao lock theo thứ tự cố định. Mỗi lần thread kế tiếp không có core là một lượt lập lịch bị mất. Lock thư viện không sụp vì thread chờ ngủ trong kernel.

RQ3. Backoff giúp TAS nhiều nhất. Ở 20 thread, TAS với sleep backoff chỉ tốn 0,31 lần thời gian của TAS. Nhưng nó đạt điều đó bằng cách cho các thread chạy lần lượt, nên độ công bằng đúng bằng $1/p$. Lock nhanh nhất là lock kém công bằng nhất. MCS công bằng tuyệt đối chừng nào mỗi thread còn có core.

RQ4. Chi phí một episode của barrier tăng khoảng 53 ns cho mỗi thread trong cụm P-core, dốc hơn khi thread tràn sang E-core, và tăng khoảng 2 000 lần khi số thread vượt số core. Spin thuần là lựa chọn tốt khi mỗi thread có core riêng. Khi vượt số core, chờ bằng `sched_yield` nhanh nhất.

RQ5. Không counter nào mở rộng. Fetch-and-add nhanh hơn vòng CAS 2,1 tới 2,7 lần từ 4 thread vì số lần CAS hỏng trên mỗi lần tăng tăng từ 0,09 ở 2 thread lên 1,59 ở 8 thread. Backoff đưa số lần hỏng về gần 0 nhưng tuần tự hoá các thread. Mutex thư viện thắng fetch-and-add khi không có việc giữa hai lần tăng, vì người giữ lock tăng nhiều lần liên tiếp trên cache line nằm sẵn trong cache của mình.

RQ6. Chưa kiểm được, vì báo cáo mới có số đo của một máy.

[Cần viết: trả lời RQ6 sau khi có số đo từ máy của các thành viên còn lại]

Bảy dự đoán viết trước khi đo bị số liệu bác bỏ toàn phần hoặc một phần. MCS không phải spinlock nhanh nhất khi số thread chưa vượt số core (D2.1). Lock chỉ bão hoà tại $p = 1 + \mathrm{OW}$ khi $1 + \mathrm{OW} \leq 4$ (D2.3), và sleep backoff vẫn giúp TAS ở nhiều thread dù có việc ngoài critical section (D3.2). Chi phí barrier không tăng theo một độ dốc duy nhất (D4.1). Mutex thư viện và vòng

CAS ở 2 thread nhanh hơn fetch-and-add (D5.1). `seq_cst` làm CAS chậm khoảng 2 lần (D5.4). Khi vượt số core, MCS mất cả tính công bằng (D3.3). Điểm chung của các dự đoán sai là chúng chỉ tính lưu lượng coherence mà bỏ qua scheduler của hệ điều hành và việc máy có hai loại core.

Từ các kết quả này, nhóm khuyến nghị ba điều. Khi số thread có thể vượt số core, dùng lock thư viện hoặc TTAS có backoff, và chỉ dùng MCS khi mỗi thread chắc chắn có core riêng. Dùng barrier spin khi số thread không vượt số core, và barrier có `sched_yield` khi có thể vượt. Dùng fetch-and-add cho counter, và chia counter theo thread khi nó nằm trên đường nóng. Việc chưa làm gồm phiên bản phân tán bằng MPI-3 RMA (phần 4 của đề) và số đo trên các máy còn lại của nhóm.

References

[1] Thomas E. Anderson. The performance of spin lock alternatives for shared-memory multiprocessors. *IEEE Transactions on Parallel and Distributed Systems*, 1(1):6–16, 1990.

[2] Debra Hensgen, Raphael Finkel, and Udi Manber. Two algorithms for barrier synchronization. *International Journal of Parallel Programming*, 17(1):1–17, 1988.

[3] Maurice Herlihy, Nir Shavit, Victor Luchangco, and Michael Spear. *The Art of Multiprocessor Programming*. Morgan Kaufmann, 2 edition, 2020.

[4] Maurice P. Herlihy and Jeannette M. Wing. Linearizability: A correctness condition for concurrent objects. *ACM Transactions on Programming Languages and Systems*, 12(3):463–492, 1990.

[5] John M. Mellor-Crummey and Michael L. Scott. Algorithms for scalable synchronization on shared-memory multiprocessors. *ACM Transactions on Computer Systems*, 9(1):21–65, 1991.

[6] Message Passing Interface Forum. *MPI: A Message-Passing Interface Standard, Version 4.1*, 2023. https://www.mpi-forum.org/docs/mpi-4.1/mpi41-report.pdf.

A Cách build và chạy lại

Yêu cầu: compiler C++20 (Apple clang, g++ hoặc clang++), `make`, Python 3; `matplotlib` và `numpy` để vẽ hình; XeLaTeX (TeX Live) để biên dịch báo cáo. Chi tiết trong `README.md`.

```
make                # biên dịch các bài kiểm tra và các benchmark vào build/
make test           # chạy mọi bài kiểm tra tính đúng đắn
make tsan           # chạy lại dưới ThreadSanitizer
make mutation       # các biến thể cố ý làm hỏng phải bị bộ kiểm tra bắt
make bench          # chạy toàn bộ đợt đo trên máy rảnh -> results/<máy>/
make figures        # vẽ lại mọi hình và bảng từ results/ -> report/figures/
make report         # biên dịch report/report.pdf
make zip            # đóng gói bài nộp
```

Trên `apple-m5`, đợt đo dài 1 giờ 25 phút 45 giây (12 152 lần chạy), gồm cả các lần chạy thử hiệu chỉnh. Lệnh đã dùng là `python3 scripts/run_bench.py --relax-yield --host apple-m5 --wait 30`, tương đương `make bench` cộng lượt tùy chọn `make bench-relax`. `make bench-dry` in lưới và thời gian ước tính mà không đo.

Nền tảng gồm macOS 27.0.1, Apple clang 21.0.0, Python 3.14.8 (chạy đo chỉ cần thư viện chuẩn). Apple clang không nhận `-fopenmp` nên không có baseline OpenMP. `matplotlib` và `numpy` cài trong một môi trường ảo (`.venv`).

[Cần viết: thời gian make bench và ghi chú cài đặt trên máy của từng thành viên còn lại.]

B Cấu trúc mã nguồn

Đường dẫn	Nội dung
`include/lock.h`	các lock của phần 1 (TAS, TTAS, biến thể backoff, MCS) và lock thư viện
`include/barrier.hpp`	sense-reversing barrier và các barrier dùng để so sánh
`include/counter.hpp`	các counter của phần 3
`include/backoff.hpp`	các chính sách chờ dùng chung cho cả ba phần
`include/platform.hpp`	cache line, lệnh chờ trong vòng spin, đồng hồ, thông tin máy
`include/workload.hpp`	`is_prime` và hàm `opaque` chống trình tối ưu
`include/bench_common.hpp`	đọc tham số, cổng xuất phát, định dạng dòng CSV
`tests/`	bài kiểm tra tính đúng đắn của ba phần
`bench/`	ba chương trình đo và `sleep_latency`
`mk/`	các mục tiêu `make` của từng phần và của việc đóng gói
`scripts/run_bench.py`	chạy toàn bộ đợt đo, ghi số liệu thô vào `results/`
`scripts/plot.py`	vẽ mọi hình, sinh mọi bảng của báo cáo từ `results/`
`scripts/machine_info.sh`	ghi lại cấu hình máy
`scripts/fake_bench.py`	chương trình giả để thử công cụ, không tạo ra số đo
`results/<máy>/`	số liệu thô của từng máy (CSV), cấu hình máy, nhật ký đợt đo
`report/`	mã nguồn báo cáo, hình và bảng sinh tự động

Bảng 9: Cấu trúc thư mục của bài nộp.

C Khai báo sử dụng công cụ AI

[Cần viết: khai báo việc sử dụng công cụ AI theo quy định của môn học (do nhóm tự viết).]